

CSE 4403: Algorithms

Lecture18

Divide and Conquer - 3

Anika Farzana

Junior Lecturer

Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: anikafarzana@iut-dhaka.edu

Convex Hull

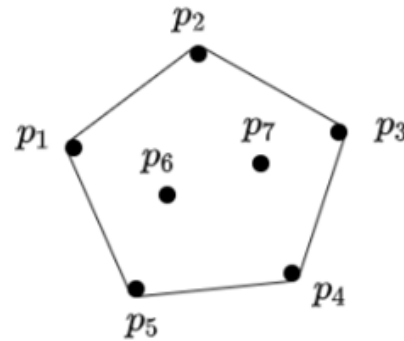
What is a convex hull?

In geometry, the **convex hull**, **convex envelope** or **convex closure** of a shape is the smallest convex set that contains it, much like a rubber band stretched tightly around a group of pins on a board.

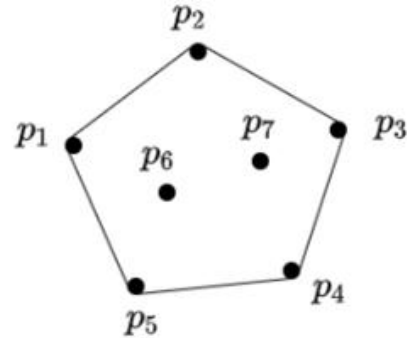
Convex Hull

What is a convex hull?

In geometry, the **convex hull**, **convex envelope** or **convex closure** of a shape is the smallest convex set that contains it, much like a rubber band stretched tightly around a group of pins on a board.



Convex Hull : Problem Statement



p_1 p_2 p_3 p_4 p_5

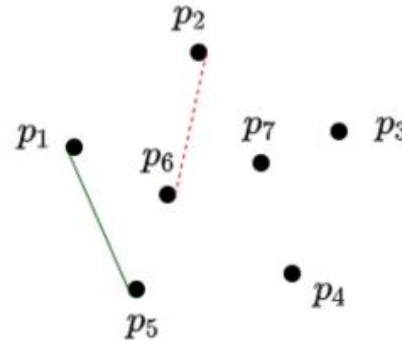
Given n points in a plane, $S = \{(x_i, y_i) \mid i = 1, 2, \dots, n\}$

- No two have the same x
- No two have the same y
- No three are in a line

Find the smallest convex polygon containing all points in S , $CH(S)$

- Sequence of boundary points
- Clockwise order

Convex Hull: Brute Force



For each pair of points

Check if the line segment is on the convex hull or not

- If rest of the points are on one side $\rightarrow$ Yes
- If rest of the points are on both sides $\rightarrow$ No

Complexity

- Total pair of points: $O(n^2)$

Convex Hull : D & C

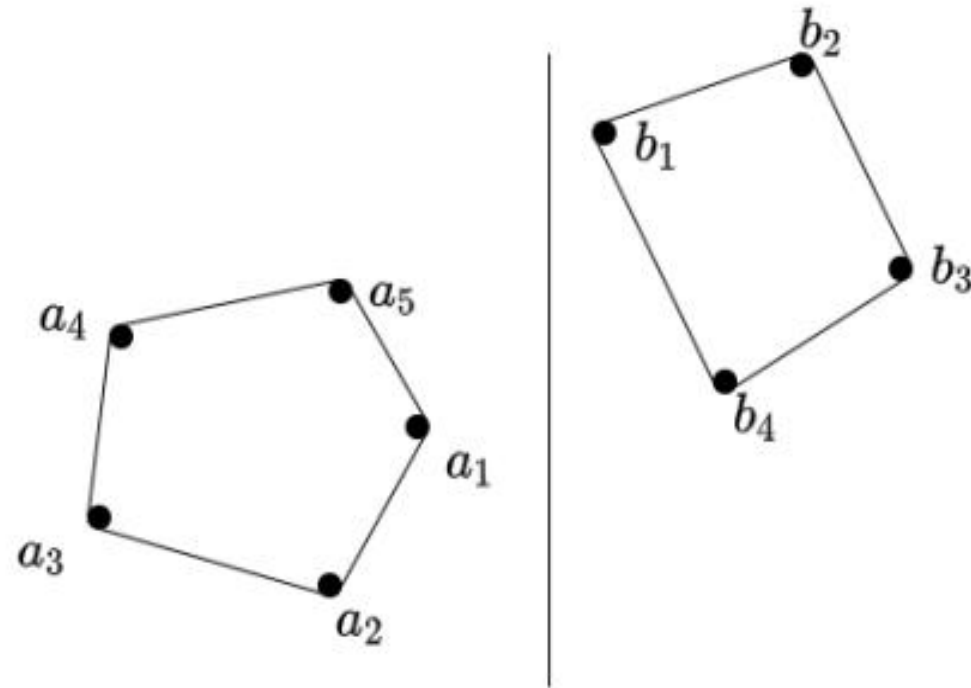
Sort the points by x -coordinates

For input set S ,

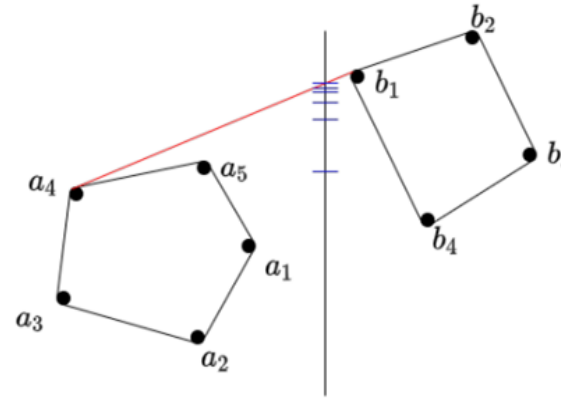
- Divide into left half A and right half B using x -coords
- Compute $CH(A)$
- Compute $CH(B)$
- Combine

Solve sufficiently smaller problem using brute force

Convex Hull : D & C



Convex Hull : D & C



Picking the max y for both is not enough

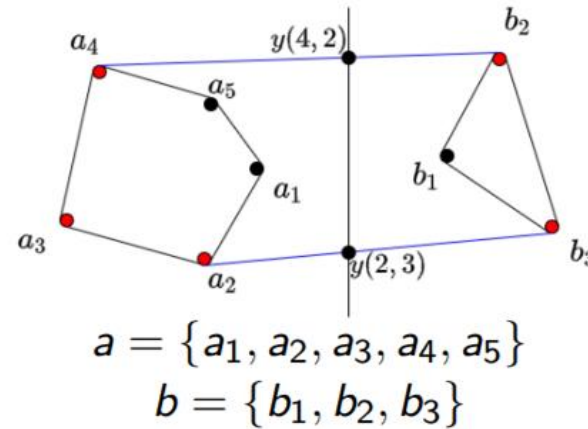
Need to maximize $y(i, j)$

Two-Finger Algorithm

- Start with line segment using rightmost of a and leftmost of b
- Move clockwise for $b \rightarrow$ Update if it improves $y(i, j)$
- Move anti clockwise for $a \rightarrow$ Update if it improves $y(i, j)$
- Repeat until $y(i, j)$ converges

Similarly find lower tangent

Convex Hull : D & C



Found two tangents? Merge.

Cut and Paste Method

- Start from left of upper tangent
- Go to the right of upper tangent
- Keep going clockwise until you reach right of lower tangent
- Move to left of lower tangent
- Keep going clockwise until you reach left of upper tangent

D & C : Time Complexity

Sort $\rightarrow O(n \log_2(n))$ (e.g. Merge Sort) or $O(n)$ (e.g. Radix Sort)

Divide and Conquer

- Divide
 - ▶ Choose partition at the center
 - ▶ $T(n) = 2T\left(\frac{n}{2}\right)$
- Merge
 - ▶ Two finger algorithm $\rightarrow O(n)$
 - ▶ Merge two lists $\rightarrow O(n)$
- Total
 - ▶ $T(n) = 2T\left(\frac{n}{2}\right) + O(n) \rightarrow$ Merge sort
 - ▶ $T(n) = O(n \log_2(n))$

Total Complexity $\rightarrow O(n \log_2(n))$

Quick Sort

```
// The quickSort function implementation
void quickSort(int arr[], int low, int high) {
    if (low < high) {

        // pi is the partition return index of pivot
        int pi = partition(arr, low, high);

        // recursion calls for smaller elements
        // and greater or equals elements
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

Lomuto's Partition function

```
// partition function
int partition(int arr[], int low, int high) {

    // Choose the pivot
    int pivot = arr[high];

    // Index of smaller element and indicates
    // the right position of pivot found so far
    int i = low - 1;

    // Traverse arr[low..high] and move all smaller
    // elements to the left side. Elements from low to
    // i are smaller after every iteration
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    // Move pivot after smaller elements and
    // return its position
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
```